

um-game-playing-strength-of-mcts-variants

6位解法 (CV 0.39)

Rank	6
Team	Vadim Timakin
Author	Vadim Timakin
Topic ID	549582
Version	Japanese Translation

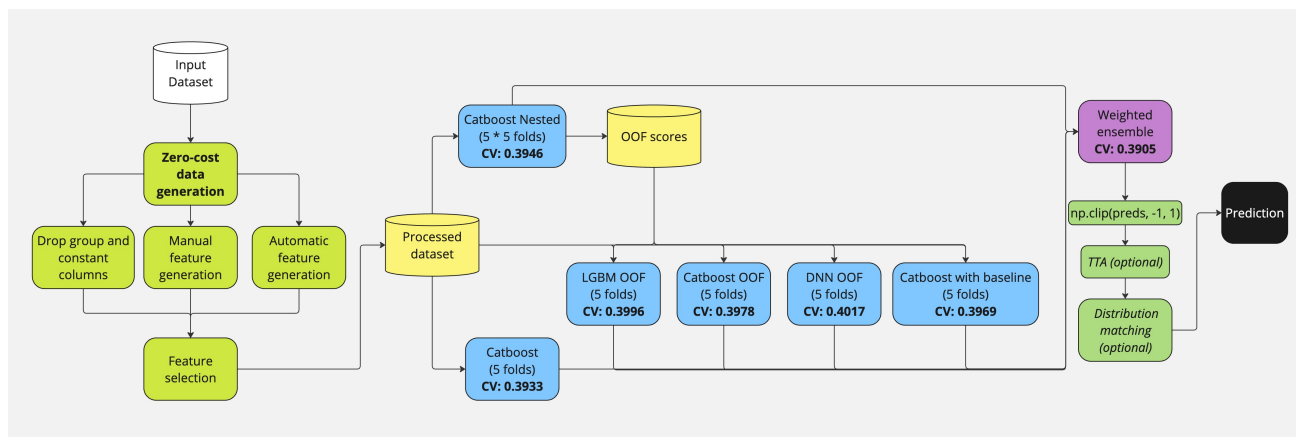
Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549582>

Retrieved with Kaggle CLI on 2026-07-03.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

1. 概要

主催者とアイデアを共有した皆さんに感謝します。私の解法は、計算不要のデータ生成、モデリング、手動・自動特徴量生成、2段階モデルのensemble、そして後述するリスクのある施策を組み合わせました。



2. 検証戦略

GameRulesetName をgroupとするGroupKFoldをbaselineにし、target、さらに agent1 を層化条件へ加えました。CV-LBの完全な相関やgap解消には至りませんでした。異なる手法・seedでより安定しました。splitは常に固定し、データ生成後も検証は元データだけで行うことが重要です。5 folds超は少し改善する一方で不安定でした。

3. データ

前処理

constant列、GameRulesetName、LudRules、EnglishRules を削除し、memoryを最適化。カテゴリへのone-hot／label／target encodingや数値のscaling／binarizing／normalizingは、DNN以外では無効でした。

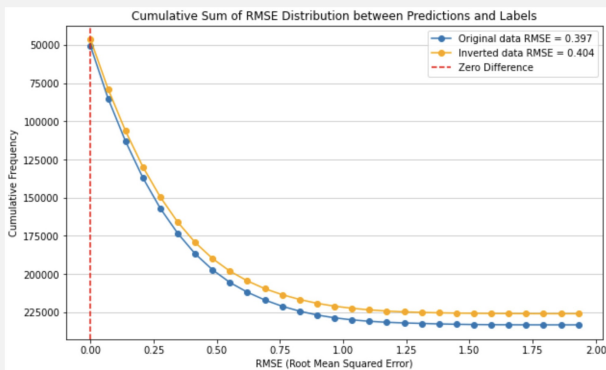
計算不要のデータ生成

agent文字列とtargetを入れ替え、当初は AdvantageP1 も反転しました。これはCV/LBを大きく改善しました。その後、AdvantageP1 は両agentがrandomにplayしたときのfirst player勝率なので論理的には反転すべきでないと感じ、反転を外すと逆に悪化しました。agentとtargetだけの交換では新しいsignalが生まれなためと解釈しました。

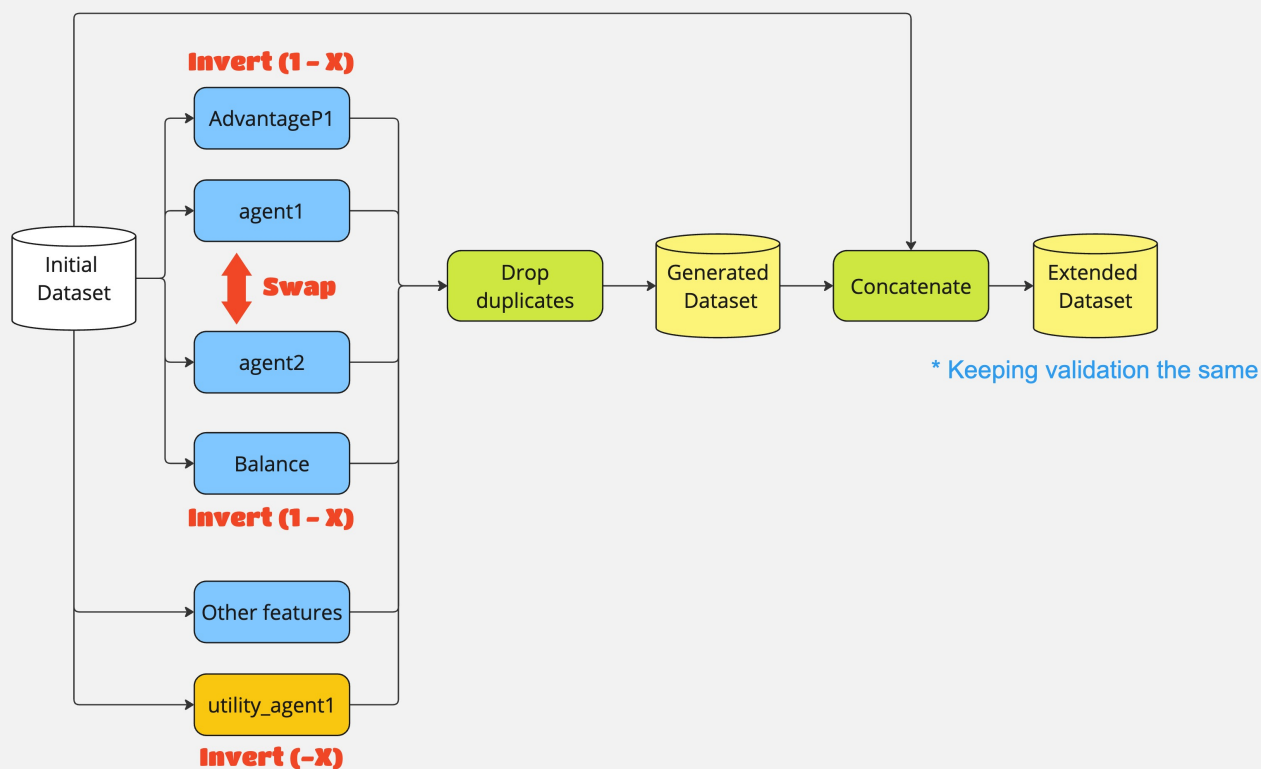
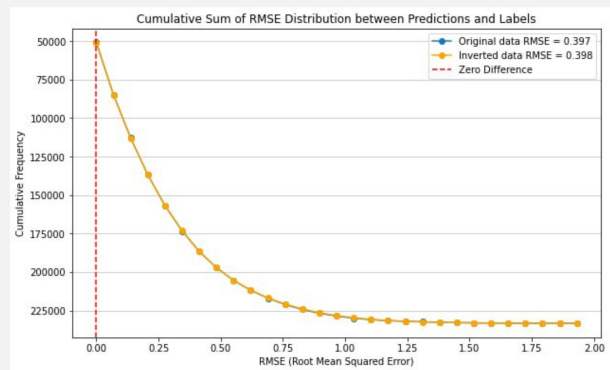
元データと反転データのOOF誤差差を埋めるため、AdvantageP1 に次いで重要な Balance も反転するとgapがほぼ消えました。他の低重要度特徴量の反転は新signalを生みませんでした。

	feature	importance
0	AdvantageP1	15.530264
1	Balance	4.956622
2	p2_selection	4.010179
3	p2_agent	2.901193
4	p2_playout	2.842518

Invert agents and Advantage



Invert agents and Advantage + Balance



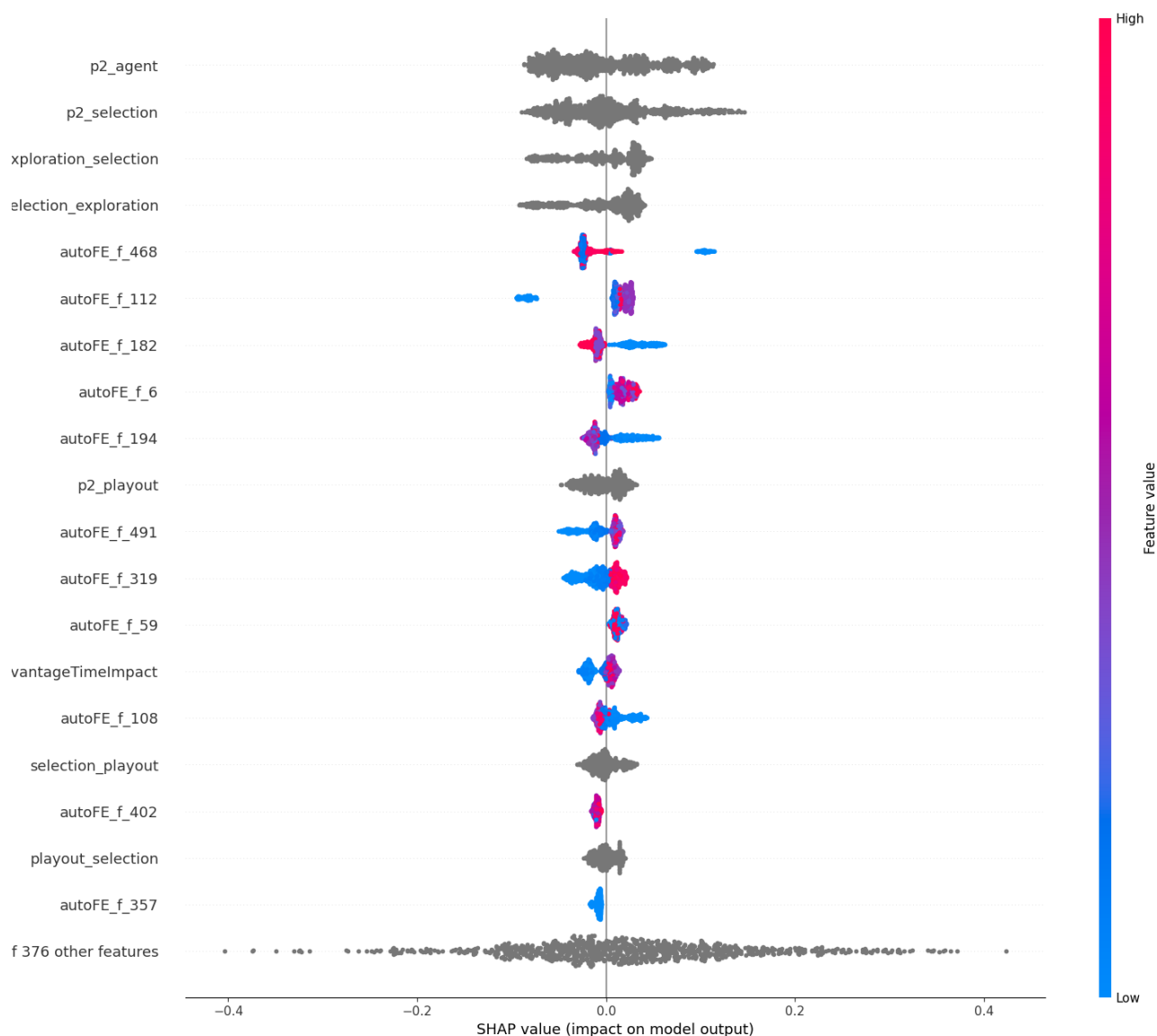
self-playなどは価値が低めでした。gap解消後はTTAで0.001~0.002改善できましたが最終提出では不使用。group splitなしの別modelでpseudo labelを生成する案も無効でした。

手動特徴量

成功したのはagent文字列からselection、exploration constant、playout、score boundsなどを抽出する特徴量、公開Notebook由来の比率・和・積、そしてagent subtype間の組合せを表すcross-agent特徴量です。LudRules / EnglishRules にTF-IDFやindex countを適用する案は無効でした。

自動特徴量生成

OpenFEを使いました。multiprocessing・dtypeのbug修正、logging調整、StratifiedKFoldからStratifiedGroupKFoldへの置換、層化・groupを保った元データ10% sample、約30時間（1 CPU）の実行、上位500特徴量選択を行いました。Public LBで不安定なgroup aggregationとfrequency特徴量を除き、NaNを-100で埋め、推論用metadataを保存しました。多くの自動生成特徴量は高い情報量を持ちました。



特徴量選択

CV importanceで約400列に絞りました。CV/LBは変わらず、memoryと速度のためです。相関、分散、group内unique数などによる追加削減は無効でした。

4. モデリング

回帰としてRMSEを最適化しました。classification+QWK、離散labelを利用したrounding threshold、feature importanceを均すregularizationはすべて悪化または不安定でした。

CatBoost

最良単体モデルで、parameterは手動調整しました。

2段階モデル

LGBM、XGB、DNNは単体でCatBoostより悪かったため、CatBoost OOFを特徴量にしました。単純な5-fold OOFはleakするため、5×5 foldsのnested CV CatBoostでOOFを作成。その後CatBoost OOFとLGBM OOFを学習し、さらにOOFを通常特徴量でなくbaseline initializationとして使うCatBoostを学習しました。

DNNは、カテゴリのembedding、数値のquantile transformer、OOF特徴量を結合したMLPです。Dropout、Hardswish、MADGRAD optimizer、RMSE lossを使用しました。game数・corner case・drawを予測するmeta-modelやXGBoostは無効でした。

ensembleはCV RMSEに対してSciPy `minimize` で正の重みを選んだweighted ensembleです。

5. 分布合わせ

trainとPublic testの分布差はCV-LB gapとLB probingから明らかでした。当初はCVを重視し、終盤に安定ensembleを無調整の安全提出とし、別のPublic最良Notebookでclipping・shifting・scalingをprobeしました。

Publicを0.421から0.417へ改善するscale+clipを見つけました。一方、最良CV ensembleへ同じ調整をしても0.424から0.423だけでした。Privateが完全新規gameの場合とPublicと交差・類似する場合の双方をcoverする狙いで

6. 最終結果

Public LBへ最も整合させた提出がPrivateでも最良でした。

Model	CV	Public	Private
CatBoost Classic	0.3933	0.426	0.432
CatBoost Nested	0.3946	0.426	0.433
CatBoost with OOF	0.3978	0.425	0.431
CatBoost OOF baseline	0.3969	0.426	0.433
LGBM with OOF	0.3996	0.426	0.432
DNN with OOF	0.4017	0.426	0.432
Best CV ensemble	0.3905	0.424	0.430
CatBoost + distribution match	0.3995	0.417	0.422

7. 追加の可能性

意味のあるLB probing、rulesを解析するLLM、test pseudo labeling、確率ゲームのrandomness処理、計算コストの高い通常のデータ生成には可能性があります。

コード：[Kaggle-MCTS-6th-place-solution](#)

補足Q&A

Aurora Rabbit: DNNが良い性能に見えます。私のDNNは重要度上位20特徴量にほぼ依存しているようです。

Vadim Timakin: CatBoost生成のOOF特徴量なしではかなり弱いモデルでした。この特徴量も当初はoverfitと低スコアを起こしましたが、高いdropout rateで解決しました。